CHANDIGARH UNIVERSITY

PROJECT REPORT

Object Oriented Programming

Course Code: 25CAH-201

Project Title

To-Do List / Task Tracker

A Command Line Application in C++

SUBMITTED BY	
Name	Jovanpreet Kaur
UID	25BCD10052
Branch	BCA Data Science
Section	25BCD-1(B)

SUBMITTED TO	
Name	Mrs. Shuchi
Designation	Professor

Declaration

I, **Jovanpreet Kaur**, UID **25BCD10052**, a student of BCA (Data Science), Section 25BCD-1(B), Chandigarh University, hereby declare that the project report entitled **"To-Do List / Task Tracker"** has been prepared by me for the course **Object Oriented Programming (25CAH-201)**. The program was written in C++, compiled and tested by me, and the outputs shown in this report were produced by running that program.

The report has been submitted to **Mrs. Shuchi, Professor**, as part of the continuous evaluation of the course. Wherever ideas or reference material from books and websites were used, they are listed in the References section at the end of this report.

Jovanpreet Kaur
UID: 25BCD10052

Acknowledgement

I would like to express my sincere gratitude to **Mrs. Shuchi** for her guidance, encouragement and clear explanation of the concepts of Object Oriented Programming throughout the semester. Her lectures on classes, inheritance, polymorphism and exception handling helped me plan this project in a proper object-oriented manner.

I am also thankful to Chandigarh University for providing the laboratory facilities and learning resources, and to my classmates and family for their constant support while I was working on this project.

Abstract

Managing daily work is a problem every student faces. This project, **To-Do List / Task Tracker**, is a command line application developed in C++ that lets a user add tasks, view them, mark them as complete and delete them. In addition, it supports tasks with deadlines, keyword search, sorting by priority, simple statistics and automatic saving of data in a text file so that nothing is lost when the program is closed.

The main aim of the project is not only to build a useful tool but also to apply the principles of object oriented programming: classes and objects, encapsulation, abstraction, inheritance, polymorphism, operator overloading, static members, friend functions, exception handling and file handling. The design is split into three classes (*Task*, *DeadlineTask* and *TaskManager*) and a small user interface in *main.cpp*. The program was compiled with GCC and tested with multiple test cases; all of them passed.

Table of Contents

1. Introduction **4**
1.1 Overview . . . 4
1.2 Background . . . 4
1.3 Objectives . . . 4
1.4 Scope of the Project . . . 4
2. Problem Statement and Feasibility **4**
2.1 Problem Statement . . . 4
2.2 Proposed Solution . . . 5
2.3 Feasibility Study . . . 5
2.4 Features of the Application . . . 5
3. Requirement Analysis **5**
3.1 Hardware Requirements . . . 5
3.2 Software Requirements . . . 6
3.3 Functional Requirements . . . 6
3.4 Non-Functional Requirements . . . 6
4. Object Oriented Concepts Used **6**
4.1 Classes and Objects . . . 7
4.2 Encapsulation and Abstraction . . . 7
4.3 Inheritance . . . 7
4.4 Polymorphism . . . 7
4.5 Constructors, Destructors and Static Members . . . 7
4.6 Operator Overloading and Friend Function . . . 8
4.7 Exception Handling . . . 8
4.8 File Handling and STL . . . 8
5. System Design **8**
5.1 System Architecture . . . 8
5.2 Data File Format . . . 8
5.3 Class Diagram . . . 9
5.4 Algorithms . . . 10
5.5 Program Flowchart . . . 10
5.6 Time Complexity . . . 10
6. Implementation **11**
6.1 Project Structure . . . 11
6.2 Task.h . . . 11
6.3 TaskManager.h . . . 13
6.4 main.cpp . . . 15
6.5 Compilation and Execution . . . 16
7. Testing and Output **16**
7.1 Test Environment . . . 16
7.2 Test Cases . . . 16
7.3 Output Screens . . . 17
7.4 Persistence Test . . . 19
7.5 Observations . . . 19
8. Limitations and Future Scope **20**
8.1 Limitations . . . 20
8.2 Future Scope . . . 20
9. Conclusion **20**
References **20**

1. Introduction

1.1 Overview

A **To-Do List** (also called a task tracker) is a tool that helps a person write down the work that has to be done, see what is still pending and tick off what has been finished. People use to-do lists for study plans, office work, shopping and household chores. Even a simple list improves productivity because the person does not have to remember everything and can concentrate on one task at a time.

This project implements such a tool as a **Command Line Interface (CLI)** application in the C++ language. The user interacts with the program by typing numbers and text in a terminal window. Although it has no graphics, a CLI program is fast, light, works on every computer and is the best way to concentrate on the logic and the object oriented design of the software.

1.2 Background

Object Oriented Programming (OOP) is a style of programming in which a program is organised as a collection of *objects*. Each object contains *data* (attributes) and the *functions* (methods) that operate on that data. C++ is one of the most popular OOP languages because it combines the speed of C with the features of classes and objects. A task list is a very natural problem for OOP, because the real world already has the object "task" with clear properties such as a title, a priority and a status, and clear actions such as creating it, completing it or deleting it.

1.3 Objectives

The objectives of this project are:

- To design and develop a working task tracker that runs in the terminal.
- To allow the user to **add**, **view**, **complete** and **delete** tasks.
- To apply the main OOP concepts taught in the course in a real, meaningful program.
- To store the data permanently in a file so that tasks are available the next time the program runs.
- To handle wrong input and errors in a safe way so that the program never crashes.
- To test the program properly and document the results.

1.4 Scope of the Project

The application is meant for a single user working on a single computer. It stores tasks in a plain text file in the same folder as the program. It supports two kinds of tasks (normal tasks and tasks with a deadline), three priority levels (Low, Medium, High) and two states (Pending and Done). Features such as reminders, multiple users, a graphical interface or synchronisation with the internet are outside the scope of this version, but they are discussed in the future scope chapter.

2. Problem Statement and Feasibility

2.1 Problem Statement

Students and working people have many small jobs to complete every day: assignments, practical files, viva preparation, fee payments, shopping and so on. Writing them on loose paper or remembering them mentally often leads to forgotten tasks and missed deadlines. Paper lists are also difficult to sort, search or update.

Problem: to develop a simple, reliable and fast software tool that can store a list of tasks, show them in an organised way, allow a task to be marked as completed or removed, and keep the data safe between two runs of the program.

2.2 Proposed Solution

The proposed solution is a menu-driven C++ program. When the program starts it loads the earlier tasks from the file `tasks.txt`, shows a menu and repeatedly asks the user what to do. Every operation is performed by the *TaskManager* class on a collection of *Task* objects. When the user chooses to exit, the manager automatically saves all tasks back to the file.

2.3 Feasibility Study

Type	Analysis	Result
Technical	Only a standard C++ compiler (GCC / MinGW / MSVC) and the C++ Standard Library are needed. No external libraries or databases are required.	Feasible
Economic	All tools used (g++, a text editor or Code::Blocks / VS Code) are free. No special hardware is required.	Feasible
Operational	The menu is simple, every option is numbered and error messages are clear, so a new user can use the program without any training.	Feasible
Schedule	The project is small and divided into three classes and one file for the interface, so it can be coded and tested within the semester.	Feasible

2.4 Features of the Application

No.	Feature	Description
1	Add task	Adds a normal task with a title and priority.
2	Add task with deadline	Adds a task that also stores a due date (DD-MM-YYYY).
3	View tasks	Shows all tasks, only pending tasks or only completed tasks in a neat table.
4	Mark complete	Changes the status of the task with the given ID to Done.
5	Delete task	Removes the task with the given ID from the list.
6	Search	Finds tasks whose title contains a keyword (not case sensitive).
7	Sort	Arranges tasks from High to Low priority.
8	Statistics	Shows total, completed and pending tasks and the percentage of progress.
9	Auto save / load	Reads tasks at start-up and writes them when the program exits.

3. Requirement Analysis

3.1 Hardware Requirements

Component	Minimum requirement
Processor	Any dual core processor (Intel / AMD) of 1.0 GHz or above
RAM	2 GB
Hard disk	50 MB of free space (the program and the data file are only a few kilobytes)
Input / Output	Keyboard and monitor

3.2 Software Requirements

Item	Details
Operating system	Windows 10/11, Linux or macOS
Language	C++ (C++14 standard or newer; `std::make_unique` is used)
Compiler	GCC (g++) 5 or higher, MinGW, Clang or MSVC. Tested with GCC 13.3 on Ubuntu 24.04
Editor / IDE	Code::Blocks, Visual Studio Code, Dev-C++ or any text editor
Libraries used	iostream, iomanip, string, vector, memory, fstream, sstream, algorithm, exception, limits, cctype (all part of the C++ standard library)

3.3 Functional Requirements

- **FR1:** The system shall allow the user to add a task with a title and a priority.
- **FR2:** The system shall allow the user to add a task with a title, a due date and a priority.
- **FR3:** The system shall display all tasks, pending tasks and completed tasks.
- **FR4:** The system shall mark a task as completed using its ID.
- **FR5:** The system shall delete a task using its ID.
- **FR6:** The system shall search tasks by keyword and sort them by priority.
- **FR7:** The system shall save tasks to a file on exit and load them on start-up.

3.4 Non-Functional Requirements

- **Usability:** a numbered menu with clear prompts and messages.
- **Reliability:** wrong input (letters instead of numbers, empty text, invalid ID) must not crash the program.
- **Performance:** every operation should finish instantly for hundreds of tasks.
- **Maintainability:** the code must be divided into classes and files so that a new feature can be added easily.
- **Portability:** only standard C++ is used, so the code runs on any operating system.

4. Object Oriented Concepts Used

The project was designed so that every important concept of the OOP syllabus is used in a natural way, not just added for show. The following table gives a quick map of where each concept appears in the program; the paragraphs after it explain them one by one.

OOP concept	Where it is used in the project
Class and object	*Task*, *DeadlineTask*, *TaskManager*, *TaskNotFoundException*
Encapsulation	Data members are `protected` / `private`; accessed only through getters and methods
Abstraction	*main.cpp* uses *TaskManager* without knowing how tasks are stored or saved
Constructor / destructor	*Task* constructor increases the counter; virtual destructor decreases it; *TaskManager* loads in the constructor and saves in the destructor
Inheritance	*DeadlineTask* inherits from *Task*; *TaskNotFoundException* inherits from *std::exception*
Polymorphism	Virtual functions *getType()*, *extraInfo()*, *serialize()*, *display()*
Static member	*Task::taskCount* and *Task::getTaskCount()*
Operator overloading	`operator<` for priority comparison, `operator<<` for printing

OOP concept	Where it is used in the project
Friend function	`operator<<` declared as friend of *Task*
Exception handling	*TaskNotFoundException* thrown by *TaskManager* and caught in *main()*
File handling	*ifstream* / *ofstream* in *loadFromFile()* and *saveToFile()*
STL and smart pointers	*vector*, *unique_ptr*, *stable_sort*, *find_if*, lambda functions

4.1 Classes and Objects

A **class** is a blueprint and an **object** is an actual item created from that blueprint. In this program *Task* is the blueprint of a task. Every time the user adds a task, one object of the class is created in memory, for example a task with ID 1, title "Submit OOP project report" and priority High. *TaskManager* is another class whose single object, created in *main()*, controls the complete list.

4.2 Encapsulation and Abstraction

Encapsulation means binding data and functions together and protecting the data from direct access. The attributes *id*, *title*, *priority* and *completed* are not public. The outside world can read them through getter functions and can change the status only through *markComplete()*. In *TaskManager* the vector of tasks is *private*, so no other part of the program can add or remove a task without going through the proper functions.

Abstraction means showing only what is necessary and hiding the details. The user-interface code in *main.cpp* simply calls *manager.addTask()* or *manager.deleteTask()*. It does not know whether the tasks are kept in a vector or an array, or how they are written to the file. If tomorrow the storage is changed to a database, only *TaskManager* needs to change.

4.3 Inheritance

Inheritance allows a new class to reuse and extend an existing class. A task with a deadline is a special kind of task, so *DeadlineTask* is derived publicly from *Task* (an "is-a" relationship). It inherits the id, title, priority and completed status and adds only one new data member, *dueDate*. This avoids repeating code and keeps the design clean.

4.4 Polymorphism

Polymorphism means "many forms". In this project it is *run-time polymorphism* achieved with virtual functions. *TaskManager* keeps a list of `unique_ptr<Task>`. Some of the pointers point to a *Task* and some to a *DeadlineTask*. When the manager calls `t->display()` or `t->serialize()`, the correct version is chosen automatically at run time, so a normal task shows "-" in the due-date column and a deadline task shows its date. The manager does not need any *if* statement to check the type. Because the base class has a **virtual destructor**, deleting an object through a base class pointer also correctly destroys the derived part.

4.5 Constructors, Destructors and Static Members

A **constructor** initialises an object. The *Task* constructor sets all attributes with a member initialiser list and increments the **static** data member *taskCount*. A static member belongs to the class, not to one object, so a single counter is shared by all tasks; the destructor decrements it and *Task::getTaskCount()* returns how many task objects exist. The *TaskManager* constructor calls *loadFromFile()* and its destructor calls *saveToFile()*, so resources are prepared and released automatically with the life of the object.

4.6 Operator Overloading and Friend Function

Operators can be given a new meaning for user-defined classes. The less-than operator `<` is overloaded in *Task* so that one task is "less than" another when it has a *higher* priority; this lets *stable_sort* arrange the list from High to Low using the expression `*a < *b`. The insertion operator `<<` is overloaded as a **friend function**: it is not a member of *Task* but it is allowed to access the protected data, so a task can be printed with `cout << task`.

4.7 Exception Handling

When the user enters an ID that does not exist, the manager does not return an error code; it **throws** a *TaskNotFoundException* object. This class inherits from *std::exception* and overrides *what()* to give a clear message. The *main()* function has a *try* ... *catch* block around the menu actions which catches the exception and prints "Error: No task found with ID ...". As a result the program keeps running instead of crashing.

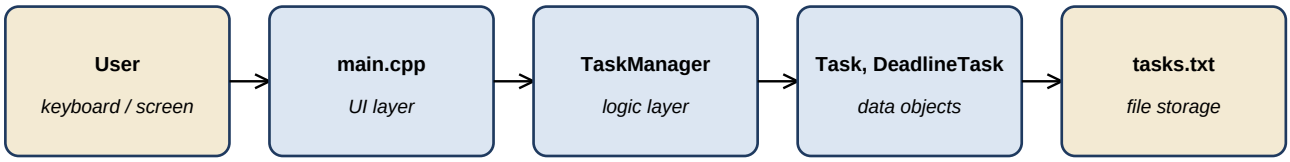
4.8 File Handling and STL

The program uses *ofstream* to write tasks and *ifstream* with *getline()* to read them. From the Standard Template Library it uses *vector* as a dynamic array, *unique_ptr* for automatic memory management (no manual *new* and *delete* and therefore no memory leaks), *find_if* to locate a task, *stable_sort* to sort while keeping equal-priority tasks in their original order, and lambda expressions as small inline comparison functions.

5. System Design

5.1 System Architecture

The software follows a simple layered architecture. The **UI layer** (*main.cpp*) shows the menu and reads the input. The **logic layer** (*TaskManager*) implements all operations. The **data layer** (*Task* and *DeadlineTask*) represents one task. Finally, the **storage** is the text file *tasks.txt*.



Each layer talks only to the layer next to it, which keeps the program easy to change.

Figure 5.1: Layered architecture of the Task Tracker

5.2 Data File Format

Tasks are stored in *tasks.txt*, one task per line, with fields separated by the pipe character (`|`). Because the pipe is the separator, the program replaces any pipe typed in a title by a slash. The structure of a line is:

```
TYPE | ID | TITLE | PRIORITY | COMPLETED | DUE_DATE
NORMAL|1|Submit OOP project report|3|1|-
DEADLINE|4|Revise Data Structures notes|3|0|08-11-2026
```

Field	Meaning
TYPE	NORMAL for *Task* or DEADLINE for *DeadlineTask*; used while loading to create the right object
ID	Unique integer number of the task
TITLE	Text of the task
PRIORITY	1 = Low, 2 = Medium, 3 = High
COMPLETED	1 = Done, 0 = Pending
DUE_DATE	Due date for a deadline task; "-" for a normal task

5.3 Class Diagram

The class diagram below shows all classes of the project with their attributes, methods and relationships. The symbols #, - and + mean protected, private and public. A hollow triangle shows inheritance, a filled diamond shows composition (the manager owns its tasks) and a dashed arrow shows dependency.

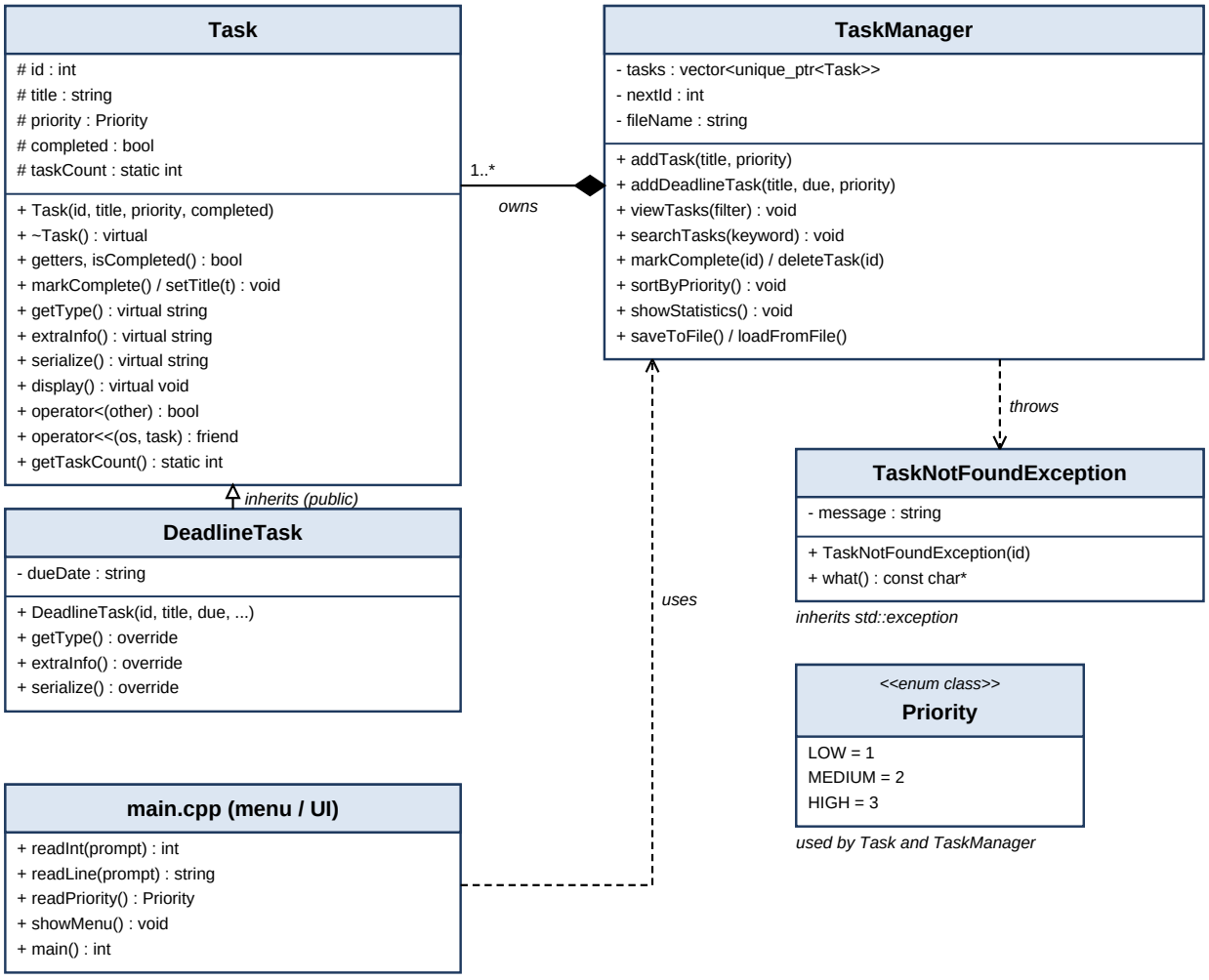


Figure 5.2: Class diagram

5.4 Algorithms

Add a task: (1) read the title and priority; (2) create a new object with the next free ID; (3) push it into the vector; (4) increase the next ID.

Mark a task as complete: (1) read the ID; (2) search the vector linearly for a task with that ID; (3) if not found, throw *TaskNotFoundException*; (4) otherwise call *markComplete()* on it.

Delete a task: (1) read the ID; (2) locate the task with *find_if*; (3) if it is missing, throw the exception; (4) otherwise erase it from the vector (the *unique_ptr* automatically frees the memory).

Load from file: (1) open the file (if it is missing, start with an empty list); (2) read each line and split it at the pipes; (3) create a *Task* or *DeadlineTask* according to TYPE; (4) keep the next ID larger than every loaded ID.

5.5 Program Flowchart

The flowchart explains the working of *main()*. The program runs in a loop; in every iteration it displays the menu, reads the choice, performs the selected operation and shows the result. The loop ends when the user selects 0, after which the tasks are saved.

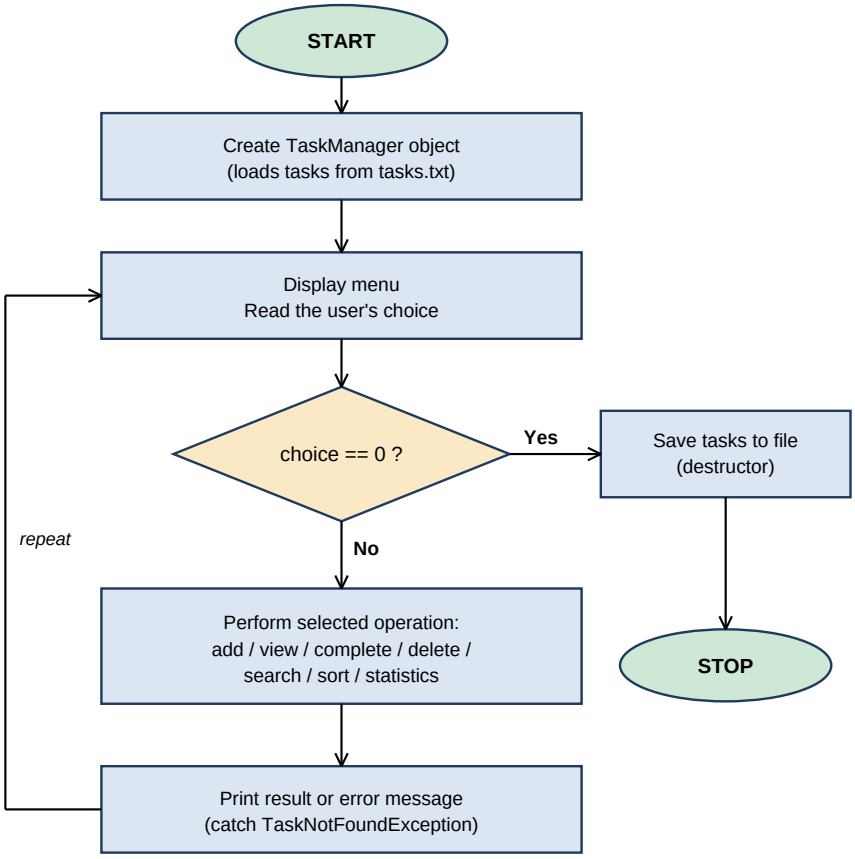


Figure 5.3: Flowchart of the program

5.6 Time Complexity

Let *n* be the number of tasks. The table shows that all operations are fast for the size of lists a person normally keeps.

Operation	Time complexity	Reason
Add task	O(1) amortised	Insertion at the end of a vector
View / statistics	O(n)	Every task is visited once
Mark complete	O(n)	Linear search for the ID
Delete task	O(n)	Linear search and shifting of elements
Search by keyword	O(n × m)	m is the length of the title being compared
Sort by priority	O(n log n)	*stable_sort*
Load / save file	O(n)	One line is processed per task

6. Implementation

6.1 Project Structure

The source code is divided into three files, which is the normal practice in C++: the classes are in header files and the program that uses them is in the main file.

File	Contents
Task.h	*Priority* enumeration, the base class *Task* and the derived class *DeadlineTask*
TaskManager.h	*TaskNotFoundException* and the *TaskManager* class (all operations and file handling)
main.cpp	Input helper functions, the menu and the main loop
tasks.txt	Data file created automatically by the program (not part of the source code)

6.2 Task.h

This header defines the priority levels, the base class *Task* and the derived class *DeadlineTask*. Observe the *protected* members, the virtual functions, the static counter, the overloaded operators and the *override* keyword in the derived class.

```
#ifndef TASK_H
#define TASK_H

#include <iostream>
#include <iomanip>
#include <string>

// ---------- Priority levels for a task ----------
enum class Priority { LOW = 1, MEDIUM = 2, HIGH = 3 };

inline std::string priorityToString(Priority p) {
    switch (p) {
        case Priority::LOW:    return "Low";
        case Priority::MEDIUM: return "Medium";
        case Priority::HIGH:   return "High";
    }
    return "Medium";
}

// ---------- Base class: a normal task ----------
class Task {
protected:
    int         id;
    std::string title;
    Priority    priority;
    bool        completed;
    static int  taskCount;              // number of Task objects alive

public:
    Task(int id, const std::string& title,
         Priority priority = Priority::MEDIUM, bool completed = false)
        : id(id), title(title), priority(priority), completed(completed) {
        ++taskCount;
    }

    virtual ~Task() { --taskCount; }
```

```
37
38      // ----- getters -----
39      int         getId()        const { return id; }
40      std::string getTitle()     const { return title; }
41      Priority    getPriority()  const { return priority; }
42      bool        isCompleted()  const { return completed; }
43
44      // ----- setters / behaviour -----
45      void markComplete()                { completed = true; }
46      void setTitle(const std::string& t)  { title = t; }
47
48      // ----- polymorphic behaviour -----
49      virtual std::string getType()   const { return "NORMAL"; }
50      virtual std::string extraInfo() const { return "-"; }
51
52      virtual std::string serialize() const {
53          return getType() + "|" + std::to_string(id) + "|" + title + "|" +
54                 std::to_string(static_cast<int>(priority)) + "|" +
55                 (completed ? "1" : "0") + "|-";
56      }
57
58      virtual void display() const {
59          std::string shown = title.size() > 26 ? title.substr(0, 23) + "..." : title;
60          std::cout << std::left
61                    << std::setw(5)  << id
62                    << std::setw(28) << shown
63                    << std::setw(10) << priorityToString(priority)
64                    << std::setw(13) << extraInfo()
65                    << (completed ? "[x] Done" : "[ ] Pending") << "\n";
66      }
67
68      // ----- operator overloading -----
69      bool operator<(const Task& other) const {          // higher priority first
70          return static_cast<int>(priority) > static_cast<int>(other.priority);
71      }
72
73      friend std::ostream& operator<<(std::ostream& os, const Task& t) {
74          os << "Task #" << t.id << " - " << t.title
75             << " (" << priorityToString(t.priority) << ")";
76          return os;
77      }
78
79      static int getTaskCount() { return taskCount; }
80  };
81
82  int Task::taskCount = 0;
83
84  // ---------- Derived class: a task with a due date ----------
85  class DeadlineTask : public Task {
86  private:
87      std::string dueDate;               // format: DD-MM-YYYY
88
89  public:
90      DeadlineTask(int id, const std::string& title, const std::string& dueDate,
91                   Priority priority = Priority::MEDIUM, bool completed = false)
92          : Task(id, title, priority, completed), dueDate(dueDate) {}
93
94      std::string getType()   const override { return "DEADLINE"; }
95      std::string extraInfo() const override { return dueDate; }
96
97      std::string serialize() const override {
98          return getType() + "|" + std::to_string(id) + "|" + title + "|" +
99                 std::to_string(static_cast<int>(priority)) + "|" +
100                (completed ? "1" : "0") + "|" + dueDate;
101     }
102 };
103
104 #endif
```

Explanation: The constructor uses an initialiser list and increments *taskCount*. *display()* prints one row of the table using *setw()* so that columns are aligned; titles longer than 26 characters are shortened with three dots. *serialize()* converts the object into one line of text for the file. *DeadlineTask* overrides *getType()*, *extraInfo()* and *serialize()* to add the due date; it does not need to rewrite *display()* because *display()* calls the virtual function *extraInfo()*.

6.3 TaskManager.h

This header contains the custom exception class and the *TaskManager* class, which is the heart of the program. The private function *findById()* is used by other functions and throws the exception when the ID does not exist.

```
#ifndef TASK_MANAGER_H
#define TASK_MANAGER_H

#include <vector>
#include <memory>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <exception>
#include <cctype>
#include "Task.h"

// ---------- Custom exception ----------
class TaskNotFoundException : public std::exception {
private:
    std::string message;
public:
    explicit TaskNotFoundException(int id)
        : message("No task found with ID " + std::to_string(id)) {}
    const char* what() const noexcept override { return message.c_str(); }
};

// ---------- Manager class ----------
class TaskManager {
private:
    std::vector<std::unique_ptr<Task>> tasks;
    int         nextId;
    std::string fileName;

    Task* findById(int id) {
        for (auto& t : tasks)
            if (t->getId() == id) return t.get();
        throw TaskNotFoundException(id);
    }

    static void printHeader() {
        std::cout << std::left
                  << std::setw(5)  << "ID"
                  << std::setw(28) << "Title"
                  << std::setw(10) << "Priority"
                  << std::setw(13) << "Due Date"
                  << "Status\n"
                  << std::string(70, '-') << "\n";
    }

public:
    explicit TaskManager(const std::string& file = "tasks.txt")
        : nextId(1), fileName(file) {
        loadFromFile();
    }

    ~TaskManager() { saveToFile(); }

    // ---------- Create ----------
    void addTask(const std::string& title, Priority p) {
        tasks.push_back(std::make_unique<Task>(nextId++, title, p));
    }

    void addDeadlineTask(const std::string& title, const std::string& due, Priority p) {
        tasks.push_back(std::make_unique<DeadlineTask>(nextId++, title, due, p));
    }

    // ---------- Read ----------
    // filter: 0 = all, 1 = pending only, 2 = completed only
    void viewTasks(int filter = 0) const {
        int shown = 0;
        printHeader();
        for (const auto& t : tasks) {
            if (filter == 1 && t->isCompleted())  continue;
            if (filter == 2 && !t->isCompleted()) continue;
            t->display();
            ++shown;
        }
        if (shown == 0) std::cout << "(no tasks to show)\n";
    }

    void searchTasks(const std::string& keyword) const {
        std::string key = toLower(keyword);
        int found = 0;
        printHeader();
        for (const auto& t : tasks) {
```

```
            if (toLower(t->getTitle()).find(key) != std::string::npos) {
                t->display();
                ++found;
            }
        }
        if (found == 0) std::cout << "(no matching tasks)\n";
    }

    void showStatistics() const {
        int done = 0;
        for (const auto& t : tasks) if (t->isCompleted()) ++done;
        int total = static_cast<int>(tasks.size());
        std::cout << "Total tasks     : " << total << "\n"
                  << "Completed       : " << done << "\n"
                  << "Pending         : " << total - done << "\n";
        if (total > 0)
            std::cout << "Progress        : " << (done * 100 / total) << "%\n";
    }

    // ---------- Update ----------
    void markComplete(int id) {
        Task* t = findById(id);           // may throw TaskNotFoundException
        t->markComplete();
    }

    void sortByPriority() {
        std::stable_sort(tasks.begin(), tasks.end(),
            [](const std::unique_ptr<Task>& a, const std::unique_ptr<Task>& b) {
                return *a < *b;           // uses overloaded operator<
            });
    }

    // ---------- Delete ----------
    void deleteTask(int id) {
        auto it = std::find_if(tasks.begin(), tasks.end(),
            [id](const std::unique_ptr<Task>& t) { return t->getId() == id; });
        if (it == tasks.end()) throw TaskNotFoundException(id);
        tasks.erase(it);
    }

    bool isEmpty() const { return tasks.empty(); }

    // ---------- File handling ----------
    void saveToFile() const {
        std::ofstream out(fileName);
        if (!out) return;
        for (const auto& t : tasks) out << t->serialize() << "\n";
    }

    void loadFromFile() {
        std::ifstream in(fileName);
        if (!in) return;                  // first run: no file yet
        std::string line;
        while (std::getline(in, line)) {
            std::stringstream ss(line);
            std::string type, id, title, pr, done, due;
            std::getline(ss, type,  '|');
            std::getline(ss, id,    '|');
            std::getline(ss, title, '|');
            std::getline(ss, pr,    '|');
            std::getline(ss, done,  '|');
            std::getline(ss, due,   '|');
            if (id.empty()) continue;

            int  tid  = std::stoi(id);
            auto prio = static_cast<Priority>(std::stoi(pr));
            bool comp = (done == "1");

            if (type == "DEADLINE")
                tasks.push_back(std::make_unique<DeadlineTask>(tid, title, due, prio, comp));
            else
                tasks.push_back(std::make_unique<Task>(tid, title, prio, comp));

            if (tid >= nextId) nextId = tid + 1;
        }
    }

private:
    static std::string toLower(std::string s) {
        std::transform(s.begin(), s.end(), s.begin(),
                       [](unsigned char c) { return std::tolower(c); });
        return s;
    }
};

#endif
```

Explanation: *viewTasks(filter)* works for all three views of the list using one parameter (0 = all, 1 = pending, 2 = completed), which avoids writing three similar functions. *sortByPriority()* passes a lambda to *stable_sort*; inside the lambda the expression `*a < *b` calls the overloaded `operator<` of *Task*. *loadFromFile()* splits each line using *stringstream* and *getline* with the delimiter '|', and creates the correct object type. The destructor calls *saveToFile()*, so tasks are saved automatically when *main()* ends.

6.4 main.cpp

The main file contains the user interface. The helper functions *readInt()*, *readLine()* and *readPriority()* validate the input so that the program is not affected by wrong data such as letters where a number is expected or an empty title.

```cpp
#include <iostream>
#include <limits>
#include "TaskManager.h"

using namespace std;

// ---------- Input helpers ----------
int readInt(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "  Invalid input. Please enter a number.\n";
    }
}

string readLine(const string& prompt) {
    string s;
    while (true) {
        cout << prompt;
        getline(cin, s);
        for (char& c : s) if (c == '|') c = '/';      // '|' is our file separator
        if (!s.empty()) return s;
        cout << "  Input cannot be empty.\n";
    }
}

Priority readPriority() {
    int p;
    do {
        p = readInt("Priority (1 = Low, 2 = Medium, 3 = High): ");
    } while (p < 1 || p > 3);
    return static_cast<Priority>(p);
}

void showMenu() {
    cout << "\n========================================\n"
         << "        TO-DO LIST / TASK TRACKER\n"
         << "========================================\n"
         << " 1. Add a new task\n"
         << " 2. Add a task with deadline\n"
         << " 3. View all tasks\n"
         << " 4. View pending tasks\n"
         << " 5. View completed tasks\n"
         << " 6. Mark a task as complete\n"
         << " 7. Delete a task\n"
         << " 8. Search tasks\n"
         << " 9. Sort tasks by priority\n"
         << "10. Show statistics\n"
         << " 0. Save and exit\n"
         << "----------------------------------------\n";
}

int main() {
    TaskManager manager("tasks.txt");
    int choice;

    do {
        showMenu();
        choice = readInt("Enter your choice: ");
        cout << "\n";

        try {
            switch (choice) {
```

```
                case 1: {
                    string title = readLine("Task title: ");
                    manager.addTask(title, readPriority());
                    cout << "Task added successfully.\n";
                    break;
                }
                case 2: {
                    string title = readLine("Task title: ");
                    string due   = readLine("Due date (DD-MM-YYYY): ");
                    manager.addDeadlineTask(title, due, readPriority());
                    cout << "Task with deadline added successfully.\n";
                    break;
                }
                case 3: manager.viewTasks(0); break;
                case 4: manager.viewTasks(1); break;
                case 5: manager.viewTasks(2); break;
                case 6: {
                    manager.markComplete(readInt("Enter task ID to mark complete: "));
                    cout << "Task marked as complete.\n";
                    break;
                }
                case 7: {
                    manager.deleteTask(readInt("Enter task ID to delete: "));
                    cout << "Task deleted.\n";
                    break;
                }
                case 8: manager.searchTasks(readLine("Search keyword: ")); break;
                case 9:
                    manager.sortByPriority();
                    cout << "Tasks sorted (High -> Low).\n";
                    break;
                case 10: manager.showStatistics(); break;
                case 0:  cout << "Saving tasks... Goodbye!\n"; break;
                default: cout << "Invalid choice. Try again.\n";
            }
        } catch (const TaskNotFoundException& e) {
            cout << "Error: " << e.what() << "\n";
        }
    } while (choice != 0);

    return 0;
}
```

Explanation: *readInt()* keeps asking until a valid number is typed; on failure it clears the error flag of *cin* and discards the wrong input. A *do ... while* loop shows the menu until the user enters 0. Each menu choice is handled by a *switch* statement placed inside a *try* block, and the *catch* block handles *TaskNotFoundException*.

6.5 Compilation and Execution

Open a terminal in the folder that contains the three files and run the following commands. The option `-Wall -Wextra` turns on compiler warnings; the project compiles without any warning.

```
g++ -std=c++14 -Wall -Wextra -o todo main.cpp
./todo            (on Windows: todo.exe)
```

7. Testing and Output

7.1 Test Environment

The program was compiled with GCC 13.3 (`g++ -std=c++14 -Wall -Wextra`) on Ubuntu 24.04 and no warnings were reported. The outputs shown below are copied from actual runs of the program; the values typed by the user are highlighted in yellow.

7.2 Test Cases

ID	Test description	Input	Expected result	Result
T1	Add a normal task	Choice 1, title, priority 3	Message "Task added successfully."	Pass
T2	Add a task with deadline	Choice 2, title, 05-11-2026, priority 2	Deadline task added	Pass

ID	Test description	Input	Expected result	Result
T3	View all tasks on an empty list	Choice 3	"(no tasks to show)"	Pass
T4	View all tasks	Choice 3	Table with all tasks	Pass
T5	Mark task complete	Choice 6, ID 1	Status changes to Done	Pass
T6	View pending / completed	Choice 4 and 5	Only matching tasks shown	Pass
T7	Mark complete with wrong ID	Choice 6, ID 99	"Error: No task found with ID 99"	Pass
T8	Delete task	Choice 7, ID 3	Task removed from list	Pass
T9	Delete with wrong ID	Choice 7, ID 42	"Error: No task found with ID 42"	Pass
T10	Search by keyword	Choice 8, "notes"	Matching task displayed	Pass
T11	Search with no match	Choice 8, "zzz"	"(no matching tasks)"	Pass
T12	Sort by priority	Choice 9, then 3	High priority tasks first	Pass
T13	Statistics	Choice 10	Total, completed, pending, progress %	Pass
T14	Empty title	Choice 1, press Enter	"Input cannot be empty." and asks again	Pass
T15	Invalid priority	Priority 7 or 5	Asks for priority again	Pass
T16	Letters in menu	x	"Invalid input. Please enter a number."	Pass
T17	Number not in menu	99	"Invalid choice. Try again."	Pass
T18	Data saved between runs	Exit with 0, run again, choice 3	Earlier tasks are still present	Pass

7.3 Output Screens

Terminal - main menu

```
==========================================
        TO-DO LIST / TASK TRACKER
==========================================
 1. Add a new task
 2. Add a task with deadline
 3. View all tasks
 4. View pending tasks
 5. View completed tasks
 6. Mark a task as complete
 7. Delete a task
 8. Search tasks
 9. Sort tasks by priority
10. Show statistics
 0. Save and exit
------------------------------------------
Enter your choice:
```

Figure 7.1: Main menu displayed when the program starts

Terminal - empty list, adding and viewing tasks

```
Enter your choice: 3

ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
(no tasks to show)

Enter your choice: 1

Task title: Submit OOP project report
Priority (1 = Low, 2 = Medium, 3 = High): 3
```

```
Task added successfully.

Enter your choice: 2

Task title: Prepare for DBMS viva
Due date (DD-MM-YYYY): 05-11-2026
Priority (1 = Low, 2 = Medium, 3 = High): 2
Task with deadline added successfully.

Enter your choice: 3

ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
1    Submit OOP project report  High      -           [ ] Pending
2    Prepare for DBMS viva      Medium    05-11-2026  [ ] Pending
3    Buy notebooks              Low       -           [ ] Pending
4    Revise Data Structures ...  High      08-11-2026  [ ] Pending
```

Figure 7.2: Viewing an empty list, adding a normal task and a deadline task, then viewing all tasks (T1 - T4)

Terminal - mark complete, filtered views and exception

```
Enter your choice: 6

Enter task ID to mark complete: 1
Task marked as complete.

Enter your choice: 4

ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
2    Prepare for DBMS viva      Medium    05-11-2026  [ ] Pending
3    Buy notebooks              Low       -           [ ] Pending
4    Revise Data Structures ...  High      08-11-2026  [ ] Pending

Enter your choice: 5

ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
1    Submit OOP project report  High      -           [x] Done

Enter your choice: 6

Enter task ID to mark complete: 99
Error: No task found with ID 99

Enter your choice: 7

Enter task ID to delete: 42
Error: No task found with ID 42
```

Figure 7.3: Marking task 1 complete, viewing pending and completed tasks, and handling wrong IDs (T5 - T7, T9)

Terminal - search, sort and statistics

```
Enter your choice: 8

Search keyword: notes
ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
4    Revise Data Structures ...  High      08-11-2026  [ ] Pending

Enter your choice: 9

Tasks sorted (High -> Low).

Enter your choice: 3

ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
1    Submit OOP project report  High      -           [x] Done
4    Revise Data Structures ...  High      08-11-2026  [ ] Pending
2    Prepare for DBMS viva      Medium    05-11-2026  [ ] Pending
3    Buy notebooks              Low       -           [ ] Pending

Enter your choice: 10

Total tasks     : 4
Completed       : 1
Pending         : 3
Progress        : 25%
```

Figure 7.4: Searching for "notes", sorting by priority and showing statistics (T10, T12, T13)

Terminal - delete a task

```
Enter your choice: 7

Enter task ID to delete: 3
Task deleted.

Enter your choice: 3

ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
1    Submit OOP project report  High      -            [x] Done
4    Revise Data Structures ...  High      08-11-2026   [ ] Pending
2    Prepare for DBMS viva      Medium    05-11-2026   [ ] Pending

Enter your choice: 0

Saving tasks... Goodbye!
```

Figure 7.5: Deleting task 3, viewing the updated list and exiting (T8, T18)

Terminal - input validation

```
Enter your choice: 1

Task title:
  Input cannot be empty.
Task title: Water the plants
Priority (1 = Low, 2 = Medium, 3 = High): 7
Priority (1 = Low, 2 = Medium, 3 = High): 2
Task added successfully.

Enter your choice: 1

Task title: Call the library
Priority (1 = Low, 2 = Medium, 3 = High): 5
Priority (1 = Low, 2 = Medium, 3 = High): 3
Task added successfully.

Enter your choice: 8

Search keyword: zzz
ID   Title                      Priority  Due Date     Status
------------------------------------------------------------------------
(no matching tasks)

Enter your choice: 99

Invalid choice. Try again.

Enter your choice: x
  Invalid input. Please enter a number.
Enter your choice: 0

Saving tasks... Goodbye!
```

Figure 7.6: Validation of empty title, wrong priority, no match, bad menu choice and letters (T11, T14 - T17)

7.4 Persistence Test

After the first run finished, the data file contained the three remaining tasks shown below. On the next run the program loaded the same three tasks, and the sorted order was also preserved because the file is written in the order of the list.

```
NORMAL|1|Submit OOP project report|3|1|-
DEADLINE|4|Revise Data Structures notes|3|0|08-11-2026
DEADLINE|2|Prepare for DBMS viva|2|0|05-11-2026
```

Figure 7.7: Contents of tasks.txt after the first run

7.5 Observations

All 18 test cases passed. The program handled every invalid input without crashing, the polymorphic functions showed the correct due-date column for each type of task, and the list was identical after restarting the program.

8. Limitations and Future Scope

8.1 Limitations

- The program is for one user and runs only in the terminal; there is no graphical interface.
- The due date is stored as text. The program does not check whether the date is valid or whether it has already passed.
- A task cannot be edited after it has been created, although the *setTitle()* function is already available in the class.
- IDs are found by linear search, which is fine for small lists but would be slower for very large ones.
- If the task with the highest ID is deleted and the program is restarted, that ID can be given again because the next ID is calculated from the largest ID present in the file.
- The data file is plain text and is not encrypted or protected by a password.

8.2 Future Scope

- **Edit task** option to change the title, priority or due date.
- **Date validation and reminders** using the <ctime> / <chrono> library, with a view of overdue tasks and tasks due today.
- **More derived classes** such as *RecurringTask* (daily or weekly) and *ShoppingTask*, taking advantage of the existing polymorphic design.
- **Categories and tags** (study, personal, work) and filtering by category.
- **Graphical interface** using Qt or a web interface, reusing the same *Task* and *TaskManager* classes.
- **Database storage** (SQLite) in place of the text file, and user accounts for multiple users.

9. Conclusion

The To-Do List / Task Tracker was successfully designed, coded in C++ and tested. It fulfils the main requirement of the project: a user can add tasks, view them, mark them as complete and delete them, and the data remains available the next time the program is started. Extra features such as deadlines, search, sort by priority and statistics make it a practical small application.

More importantly, the project gave practical experience of Object Oriented Programming. Dividing the problem into the classes *Task*, *DeadlineTask* and *TaskManager* made the program easy to understand, test and extend. Concepts such as encapsulation, inheritance, polymorphism, operator overloading, exception handling and file handling, which were earlier studied only in theory, were applied together in one working program. The project also improved my skills in planning a design, writing clean code, debugging and documenting the work.

In conclusion, the project shows that even a small program becomes more organised, reusable and reliable when it is built with the object oriented approach, and it can serve as a base for larger applications in the future.

References

1 Robert Lafore, *Object-Oriented Programming in C++*, 4th Edition, Sams Publishing.

2 E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill Education.

3 Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.

4 C++ reference documentation: https://en.cppreference.com

5 Course lecture notes and laboratory manual of Object Oriented Programming (25CAH-201), Chandigarh University.